

In This Solution By CloudAge Global, A Large Language Model (LLM) Is Hosted On Amazon SageMaker. The Intuitive Features And Robust Infrastructure Of SageMaker Are Used To Deploy Sophisticated AI Models Reliably And Securely.

Users Of All Skill Levels Can Efficiently Manage And Operate Their LLMs, Making The Whole Process Straightforward And User-Friendly. Amazon SageMaker Studio, A Unified Machine Learning Integrated Development Environment (IDE), Streamlines The Model's Development, Training, Testing, And Deployment From One Central Location.

A SageMaker Studio Notebook Is Used To Deploy The Model As A Hosted Inference On The SageMaker Endpoint. AWS Lambda Functions Are Used To Interact With The SageMaker Hosted Inference, While Amazon API Gateway Bridges Requests From The Frontend To Lambda Securely And At Scale.

Below Are The Detailed Steps For CloudAge Global's Deployment:

1. After Logging Into The AWS Console, Go To Amazon SageMaker AI.
 - 1a. In SageMaker AI, Go To Studio And Create A User With Default Settings.
 - 1b. Launch Studio To Create A JupyterLab Space With Default Settings.
Example: (Instance: ml.t3.medium, Storage: 15GB).
 - 1c. Open Jupyter Notebook And Import “jupyter_notebook.ipynb” File From Desktop.
 - 1d. Run The Code Starting From Step 2 Until Step 3 At Least, Then Continue Running All Code Sections.

- 2a. Now, Go To AWS Lambda And Create A Function. While Configuring, Choose Python 3.11.
- 2b. Ensure The IAM Role Has The AmazonSageMakerFullAccess Policy Attached To API Gateway Or As Required By Your Organisation's Policies.
- 2c. Copy The Code From Desktop "lambda_function.py" File And Paste It In The Lambda Code Editor.
- 2d. Go To Configurations → Environment Variables → Add Environment Variable.
- 2e. Key → ENDPOINT_NAME, Value → "hf-llm-falcon-7b-instruct-bf16-2025-07-10-07-10-16-076". Best Practice Is To Copy The Name Directly From The Endpoint In SageMaker Studio.
- 2f. Deploy The Lambda Function.

3. Next, Go To AWS Console, Search API Gateway, And Choose REST API.

- 3a. Click On Import And Upload "generative-ai-api-prod-swagger-apigateway.json".
- 3b. Review Resources And Stages.
- 3c. From The Dropdown In The POST Resource, Click On "Integration Request" And Replace The Lambda Function With Your Deployed Lambda.
- 3d. Confirm Any Code Changes Made Locally Have Been Applied.
- 3e. Deploy The API And Create A New Stage Named According To Your Organisation's Naming Conventions.

The GenAI Backend Is Now Ready. Let's Prepare The Frontend.

4. On Your Desktop:

- 4a. Edit "index.html" And Change → `var apiGatewayUrl = "{API_GATEWAY_POST_URL}"`.

- 4b. Upload “logo.jpeg” And “index.html” To The S3 Bucket You Will Use In CloudFront.
5. Go To AWS CloudFront And Create A Distribution, Specify Origin As S3, And Choose The Bucket Containing “index.html”.
- 5a. Go To CloudFront Distribution → General → Edit → Set Default Root Object To “index.html”.
- 5b. Go To Your CloudFront Distribution → Origins → Check “cloudage-gen-ai...” → Edit → Copy The S3 Bucket Policy.
- 5c. Go To The S3 Bucket Containing “index.html” And Edit The Bucket Policy.
- 5d. Or Paste The S3 Policy From Desktop, Remembering To Update The ARN.
- 5e. Wait Until The CloudFront Distribution Deploys To All Edge Locations (This May Take Around 10 Minutes Depending On The Size Of The Web Application).
- 5f. Go To CloudFront Distribution → Invalidations → Create Invalidation And Paste “/*”.
- 5g. Once Deployed, Go To The URL (Distribution Domain Name).

The Sample Application, Hosted In Amazon S3 And Delivered Via CloudFront, Invokes The API Gateway Which Calls The Lambda Function, Completing The Secure And Scalable Workflow.

To Update Or Replace The LLM, Simply Deploy A New Model On The SageMaker Endpoint, Update The Lambda Code To Point To The New Endpoint, And Validate Using The Sample Application.

Great Work, Project Completed. All Praise Be To Almighty GOD Alone.